

1 Backpropagation, Justifying Gradients

Why does a step in GD always result in a local decrease?

- $f(\mathbf{x} + \varepsilon) = f(\mathbf{x}) + \varepsilon^T \nabla_{\mathbf{x}} f(\mathbf{x}) + \mathcal{O}(\|\varepsilon\|^2)$
- Noting that ε is “small”, raising it to the power of 2 disappears in comparison to no power
- $\varepsilon = -\eta \nabla_{\mathbf{x}} f(\mathbf{x})$, where η is the learning rate < 1
- $f(\mathbf{x} - \eta \nabla_{\mathbf{x}} f(\mathbf{x})) = f(\mathbf{x}) - \eta (\nabla_{\mathbf{x}} f(\mathbf{x})^T \nabla_{\mathbf{x}} f(\mathbf{x})) + \mathcal{O}(\|\eta \nabla_{\mathbf{x}} f(\mathbf{x})\|^2)$
- $(\nabla_{\mathbf{x}} f(\mathbf{x})^T \nabla_{\mathbf{x}} f(\mathbf{x})) \geq 0$ (since this is like squaring), hence the middle term is non-negative, and thus $f(\mathbf{x} - \eta \nabla_{\mathbf{x}} f(\mathbf{x})) \lesssim f(\mathbf{x})$

1.1 SGD

- Batch GD: $\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i(\theta)$, $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$. SGD: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_i(\theta)$, $E_i[\nabla_{\theta} \mathcal{L}_i(\theta)] = \frac{1}{N} \sum_{j=1}^N \nabla_{\theta} \mathcal{L}_j(\theta)$
- SGD better than mini-batch if gradient of examples are similar
- SGD more noisy and less parallelizable
- The best mini-batch size **B**:
 - Large enough so updates are not noisy
 - Large so computation parallelizable
 - Small, so redundancy can be exploited

So, we set it in practice by setting it as large as what memory prefers, since memory limits are often reached before the theoretical sweet spot. Keep in mind, larger $\rightarrow$ more mem.

1.2 Learning Rate Schedule

Making a learning rate schedule involves planning:

- Constant (always the same): $\eta(t) = \eta$
- Piecewise constant: $\eta(t) = \eta_i$ if $t_i \leq t \leq t_{i+1}$
- Exponential decay: $\eta(t) = \eta_0 e^{-\lambda t}$

Smaller $\eta \rightarrow$ smaller steps near min. $\rightarrow$ less oscillation. Choice of such a schedule is a hyperparameter. Want higher η at start.

1.3 ADAM

- Momentum is responsible for direction
 - Consistent gradients accumulate and $\mathbf{v}_t$ grows.
 - Noisy gradients cancel and $\mathbf{v}_t$ shrinks.
 - **Bad** for very sparse data!
- RMSProp ensures rarely updated dimensions are updated more
 - Prevent over-updating of parameters with strong or frequent updates.
 - Encourage learning of parameters with rare but informative updates.

Adam combines momentum and RMSProp:

- $\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + (1 - \beta) \mathbf{g}_t$
 - w/ a proper moving avg, scale is consistent and easy to interpret and combine w/ other methods. Naïve omits $(1 - \beta)$ term
- $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) \mathbf{g}_t \odot \mathbf{g}_t$

Draft: $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\mathbf{s}_t + \varepsilon}} \odot \mathbf{v}_t$ (no momentum: $\mathbf{g}_t$ in place of $\mathbf{v}_t$)

Problem: $\mathbf{v}_t = (1 - \beta^t) \mathbf{g}$, and $(1 - \beta^t)$ is very small at iteration 1, and then increases to 0.19 in 2nd iteration. Hence, $\mathbf{v}_t$ is **biased toward 0 in early steps**. It does level out eventually, but we need to **correct this bias**: $\hat{\mathbf{v}}_t \leftarrow \frac{\mathbf{v}_t}{1 - \beta^t}$. As $t \rightarrow \infty$, $1 - \beta^t \rightarrow 1$, so $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta^t} \approx \mathbf{v}_t$. Yes, this applies to $\mathbf{s}_t$: $\hat{\mathbf{s}}_t \leftarrow \frac{\mathbf{s}_t}{1 - \gamma^t}$. That's all we need.

Conclusion:

1. Momentum: $\mathbf{v}_t \leftarrow \beta \mathbf{v}_{t-1} + (1 - \beta) \mathbf{g}_t$
2. RMSProp: $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + (1 - \gamma) (\mathbf{g}_t \odot \mathbf{g}_t)$
3. Bias corr: $\hat{\mathbf{v}}_t \leftarrow \frac{\mathbf{v}_t}{1 - \beta^t}$, $\hat{\mathbf{s}}_t \leftarrow \frac{\mathbf{s}_t}{1 - \gamma^t}$
4. Update: $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{\sqrt{\hat{\mathbf{s}}_t + \varepsilon}} \odot \hat{\mathbf{v}}_t$

These hyperparameters are good defaults:

$\beta = 0.9$, $\gamma = 0.999$, $\eta = 0.001$, $\varepsilon = 10^{-8}$.

The good: Smooths updates. Scales updates. Corrects bias towards 0 early. Good default hyperparameters.

Adam memory requirements. Tripled. $\mathbf{v}_t$, $\mathbf{s}_t$ must be stored, making two “copies” of θ_t .

2 CNNs / Batch Normalization

Fully connected means regular

- $H_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} V_{a,b} X_{(i+a),(j+b)}$

Goals: (1) Translation invariance, (2) Locality. Objects can vary in appearance. This is why vision is hard. Convolution is a linear operation; need non-linearity.

THE BIAS IS A SCALAR PER CHANNEL

- **TRANS.IV:** Recognize same feature in any region of image
 - How: same filter everywhere in image; $V_{a,b}$ and u independent on (i, j) .
- **LOC:** Operation only depends on small region of image
 - How: Each output value depends on small

$(2\Delta - 1) \times (2\Delta - 1)$ region of input, where Δ does not span full image.

2.1 Convolution Operation

Left: input, right: kernel.

$$\begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix} * \begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}.$$

Output height: $H_{\text{out}} = H_{\text{in}} - H_{\text{ker}} + 1$

Side length of the receptive field marginalized:

- $R_i = R_{i-1} + (K_i - 1) S_{i-1}$
 - R_i : receptive field at layer i , $R_0 = 1$
 - K_i : kernel width/height at layer i
 - S_i : cumulative stride $= \prod_{j=1}^i s_j$ where s_j is the stride of that layer, $s_0 = 1$ and $S_0 = 1$.
- **Padding does not impact result.** Same logic for pooling

2.1.1 Why use a larger initial convolution?

- Raw high-res images have very little info in tiny regions (3×3).
- A larger internal kernel provides several benefits:
 - Captures useful structure early.
 - Handles high-resolution input without losing information
 - Reduces resolution w/ pooling for efficiency

2.2 Padding

Conventions: use odd kernel sizes. Make them square. **Pad.**

Padding required so output size = input size: Add (kernel size - 1)/2 pixels on each side.

Why? Preserve edge info otherwise lost. Prevents feature maps from shrinking, enabling deeper networks.

Stride convention: Kernels exactly don't overlap but touches full input.

2.3 Channels

RGB has three channels, hence (d : output channel):

$$H_{i,j,d} = u_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_{c=1}^3 V_{a,b,c,d} X_{(i+a),(j+b),c} - \text{One kernel for each input-output channel combination.}$$

2.4 Calculating Output Height

Input $IH \times IW$, padding p , kernel $KH \times KW$, stride s for both directions. Then: $OH = \left\lfloor \frac{IH + 2p - KH}{s} \right\rfloor + 1$

2.5 Parameter Count

- WEIGHTS = $FW \times FH \times \text{no. input ch} \times \text{no. output ch}$
- BIASES = no. output ch
- *If must find padding, remove floor & ceil padding*

2.6 Pooling (“Downsampling”)

Max pooling is the most common. **Always stride by pooling size, ensure no overlap. Pooling is parameterless.**

Effects: (1) reduce parameters & computations. (2) Extract dominant features regardless of exact positions. (3) Helps achieve invariance to small translations and distortions in input.

2.7 Performing Classification

Flatten eventually. Convolution is the learned feature extractor, **MLP** is the classification head.

2.8 Batch Normalization

We standardize features before input is fed to model. We may want to rescale hidden activations, but how do we do that? Consider a hidden layer with activations

- $\mathbf{x} \in \mathbb{R}^{B \times D}$.
1. Compute batch's mean activation:
$$\mu_i = \frac{1}{|B|} \sum_{b=1}^B x_{b,i}, i = 1 \dots D$$
 2. Compute batch variance:
$$\sigma_i^2 = \frac{1}{|B|} \sum_{b=1}^B (x_{b,i} - \mu_i)^2, i = 1 \dots D$$
 3. Perform normalization:
$$\text{BN}(x_{b,i}) = \gamma_i \odot \frac{x_{b,i} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}} + \beta_i, i = 1 \dots D, b = 1 \dots B$$

γ and β are **learned** parameters w/ same dimensionality as $\mathbf{x}$:

- γ controls **std. dev** of distribution after normalization.
- β controls **mean** of the distribution after normalization

Must control std. dev & mean since activations are **forced** to have zero mean and unit variance, limiting model's expressive power.

Batch norm is another network layer that gets inserted between a hidden layer and the next hidden layer.

2.8.1 Training vs. Inference

During training:

- Compute μ and σ for each batch of data.
- Compute exponential moving averages of μ and σ
- $\mu_{\text{ema}} \leftarrow p \mu_{\text{ema}} + (1 - p) \mu$

During inference: use pre-computed EMA μ and σ from training.

2.8.2 Batch Normalization of Images

Each image has shape $\text{batch} \times \text{ht} \times \text{wt} \times \text{channel}$. BatchNorm computes mean μ_B and std σ_B by averaging over batch, height, and width dimensions.

Features at different locations are normalized identically. This can detect features regardless of their position.

Example: 32 RGB images, each 64×64 .

- Input shape [32, 64, 64, 3]
- For red channel: compute μ_B and σ_B across all 32 images and 64×64 positions per image.

2.8.3 Rationale

Original claim: BatchNorm works by reducing internal covariate shift. Apparently, that's not correct. **It moves the activations towards non-saturated regions of the non-linearity.** For ReLU or sigmoid, shifts activations back to the learned shift. Smooths the optimization landscape. Has regularization effect due to noisy estimates of mean and variance.

Concern: Model output per example depends on batch selected. Violates expectation that predictions are independent of batch.

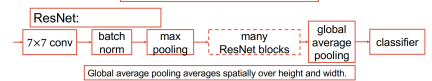
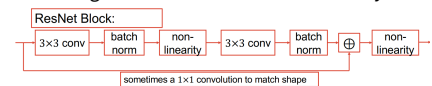
2.9 Residual Connections

- New layers shouldn't distort features learned by previous layers.
- New layers should allow gradients to propagate effectively.
- Example: adding a layer that always outputs zero.

Solution: add a **skip** connect that preserves original input: output = $f(x) + x$

In the convolution case, when $f(x)$ changes shape (channels, height, or width), apply a 1×1 convolution on the residual path to match dimensions, w/ the correct padding and stride.

- Kernel size: 1×1 . Input channels: match residual input.
 - Padding: 0 (unneeded), output channels: match main path output
 - **Stride: match main path stride**
- 2.9.1 Rationale**
- Enables training much deeper model
 - Smoothens optimization landscape
 - Allow gradients to flow more effectively



3 Sequences / RNN

Autoregressive: An autoregressive (AR) model is a statistical and machine learning technique that predicts future values in a sequence based on its own previous values

- Sequences decomposable as $p(x_1, \dots, x_T) = p(x_1) p(x_2|x_1) p(x_3|x_1, x_2)$
- New samples via autoregressive sampling: $x_1 \sim p(x_1), x_2 \sim p(x_2|x_1)$
- RNN accepts variable-length prefix, prediction depends on **all** prev. elements, fixed no. of parameters.

3.1 RNNs

One hidden layer: $\tilde{\mathbf{h}} = \sigma(\mathbf{W}_h \tilde{\mathbf{x}} + \tilde{\mathbf{b}}_h)$ and $\mathbf{o} = \mathbf{W}_o \tilde{\mathbf{h}} + \tilde{\mathbf{b}}_o$



Autoregressive prediction:

1. $\mathbf{x}_1 \rightarrow \mathbf{h}_1 \rightarrow \mathbf{o}_1 \rightarrow \text{predict } \mathbf{x}_2$
2. $(\mathbf{x}_2 \text{ and } \mathbf{h}_1) \rightarrow \mathbf{h}_2 \rightarrow \mathbf{o}_2 \rightarrow \text{predict } \mathbf{x}_3$
3. Repeat for full sequence.

3.2 Backprop Through time

Recall these gradients

- $\mathbf{z}^L = \mathbf{W}^L \mathbf{a}^{L-1} + \mathbf{b} \approx \mathbf{A}^{L-1} (\mathbf{W}^{(1)})^T + \mathbf{1}_N (\mathbf{b}^{(L)})^T$
- $\nabla_{\mathbf{W}^L} \mathcal{L} = (\nabla_{\mathbf{z}^L} \mathcal{L}) (\mathbf{a}^{L-1})^T \approx (\nabla_{\mathbf{z}^L} \mathcal{L})^T \mathbf{A}^{L-1}$
- $\nabla_{\mathbf{a}^{L-1}} \mathcal{L} = (\mathbf{W}^L)^T (\nabla_{\mathbf{z}^L} \mathcal{L}) = (\nabla_{\mathbf{z}^L} \mathcal{L}) \mathbf{W}^L$
- $\nabla_{\mathbf{z}^{L-1}} \mathcal{L} = (\nabla_{\mathbf{a}^{L-1}} \mathcal{L}) \odot \sigma'(\mathbf{z}^{L-1})$

Forward pass

- $\tilde{\mathbf{z}}_t = \mathbf{W}_{hx} \mathbf{x}_t + \mathbf{W}_{hh} \mathbf{h}_{t-1} + \mathbf{b}_h$
- $\tilde{\mathbf{h}}_t = \sigma(\tilde{\mathbf{z}}_t)$
- $\tilde{\mathbf{o}}_t = \mathbf{W}_o \tilde{\mathbf{h}}_t + \tilde{\mathbf{b}}_o$
- $L = \frac{1}{T} \sum_{t=1}^T L_t(\tilde{\mathbf{x}}_t, \tilde{\mathbf{o}}_t)$

Backward pass

- $\nabla_{\mathbf{W}_o} L = \sum_{t=1}^T (\nabla_{\mathbf{o}_t} L) \cdot (\mathbf{h}_t)^T$
- $\nabla_{\mathbf{W}_{hx}} L = \sum_{t=1}^T (\nabla_{\mathbf{z}_t} L) \cdot (\mathbf{x}_t)^T$

- $\nabla_{\mathbf{W}_{hh}} L = \sum_{t=1}^T (\nabla_{\mathbf{z}_t} L) \cdot (\mathbf{h}_{t-1})^\top$
 - $\nabla_{\mathbf{z}_t} L = \nabla_{\mathbf{h}_t} L \odot \sigma'(\mathbf{z}_t)$
 - $\nabla_{\mathbf{h}_t} L = \mathbf{W}_0^\top (\nabla_{\mathbf{o}_t} L) + \mathbf{W}_{hh}^\top (\nabla_{\mathbf{z}_{t+1}} L)$
- 3.2.1 Backpropagation through time**
- $\frac{\partial L}{\partial \mathbf{W}_{hh}} = \frac{1}{T} \sum_{t=1}^T \frac{\partial L(\mathbf{x}_t, \mathbf{o}_t)}{\partial \mathbf{W}_{hh}} = \frac{1}{T} \sum_{t=1}^T \frac{\partial L(\mathbf{x}_t, \mathbf{o}_t)}{\partial \mathbf{o}_t} \frac{\partial \mathbf{o}_t}{\partial \mathbf{W}_{hh}} + \frac{\partial \mathbf{z}_t}{\partial \mathbf{W}_{hh}} \frac{\partial \mathbf{z}_t}{\partial \mathbf{W}_{hh}}$ (we get this recurrence relation)

Where we define $a_t = \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$, $b_t = \frac{\partial \sigma(\mathbf{z}_t)}{\partial \mathbf{W}_{hh}}$, $c_t = \frac{\partial \sigma(\mathbf{z}_t)}{\partial \mathbf{h}_{t-1}}$ we get $a_t = b_t + c_t a_{t-1}$ as the recurrence relation. We can unwrap the recurrence relation:

- $a_0 = 0$
- $a_1 = b_1 + c_1 a_0 + b_1$
- $a_2 = b_2 + c_2 a_1 + b_2 + c_2 b_1$
- $a_3 = b_3 + c_3 a_2 = b_3 + c_3 b_2 + c_3 c_2 b_1$

Pattern-matching, we get:

- $a_t = b_t + \sum_{j=1}^{t-1} \left(\left(\prod_{i=j+1}^t c_i \right) b_j \right)$

Solution to the recurrence relation

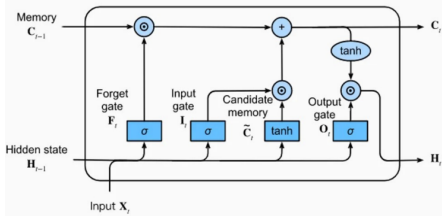
- $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \frac{\partial \sigma(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{W}_{hh})}{\partial \mathbf{W}_{hh}} + \sum_{j=1}^{t-1} \left(\prod_{i=j+1}^t \frac{\partial \sigma(\mathbf{z}_i)}{\partial \mathbf{h}_{i-1}} \right) \frac{\partial \sigma(\mathbf{z}_j)}{\partial \mathbf{W}_{hh}}$

If σ was linear, then we can unwrap the entire thing:

$$\frac{\partial \mathbf{h}}{\partial \mathbf{W}_{hh}} = \mathbf{h}_{t-1} + \sum_{j=1}^{t-1} \left(\prod_{i=j+1}^t \mathbf{W}_{hh} \right) \mathbf{h}_{j-1}. \text{ Notice the exploding/vanishing gradient problem.}$$

3.3 LSTM

- **Forget gate:** $\mathbf{F}_t = \sigma(\mathbf{W}_{xf} \mathbf{x}_t + \mathbf{W}_{hf} \mathbf{h}_{t-1} + \mathbf{b}_f)$
- **Input gate:** $\mathbf{I}_t = \sigma(\mathbf{W}_{xi} \mathbf{x}_t + \mathbf{W}_{hi} \mathbf{h}_{t-1} + \mathbf{b}_i)$
- **Output gate:** $\mathbf{O}_t = \sigma(\mathbf{W}_{xo} \mathbf{x}_t + \mathbf{W}_{ho} \mathbf{h}_{t-1} + \mathbf{b}_o)$
- **Candidate value:** $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_{xc} \mathbf{x}_t + \mathbf{W}_{hc} \mathbf{h}_{t-1} + \mathbf{b}_c)$



No repeated matrix multiplications or non-linearity.

- Cell state: $\mathbf{c}_t = \mathbf{F}_t \odot \mathbf{c}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{c}}_t$
- Gradient flow: $\frac{\partial L}{\partial \mathbf{c}_t} = \frac{\partial L}{\partial \mathbf{c}_{t+1}} \odot \mathbf{F}_{t+1}$

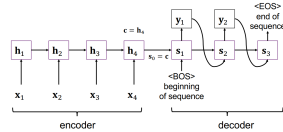
3.4 Encoder-Decoder

Use two RNNs: encoder and decoder. Encoder reads and summarizes input seq. Decoder generates output sequence step-by-step.

How do we feed context $\mathbf{c}$ into decoder RNN:

$$\mathbf{s}_t = g(\mathbf{y}_{t-1}, \mathbf{s}_{t-1}, \mathbf{c})$$

1. Concat $\mathbf{c}$ w/ input at each time step
2. Initialize the decoder's hidden state with $\mathbf{c}$, $\mathbf{s}_0 = \mathbf{c}$
3. Use attention mechanism



4 Attention

- **Key/Value** (like a dict), **Query** (used to retrieve values)
 - $\mathbf{q} \in \mathbb{R}^{d_q}$, $\mathbf{k}_i \in \mathbb{R}^{d_k}$, $\mathbf{v}_i \in \mathbb{R}^{d_v}$, $i = 1 \dots T$; no. keys = no. values, often $d_q = d_k \neq d_v$
- **Comparison:** computes similarity between query and each key. $a_i = \alpha(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}$, $\mathbf{a} \in \mathbb{R}^T$
- **Reduction:** returns a weighted combination of values using similarity scores (e.g. max, sum, avg): $f(\mathbf{V}, \mathbf{a}) \in \mathbb{R}^{d_v}$

4.1 Seq2Seq

- **Query:** $\mathbf{s}_{t-1}$, prev. decoder state (dealing w/ one query)
- **K/V:** $\mathbf{k} = \mathbf{v}_i = \mathbf{h}_i$ (encoder states are the hidden values)
- **Comparison:** $\alpha(\mathbf{q}, \mathbf{k}_i) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}_i) = a_i$ ($\mathbf{W}$ and $\mathbf{w}$ are learnable)
- **Reduction:** $\mathbf{c}_t = \sum_{i=1}^T \text{softmax}(\mathbf{a}_i) \mathbf{h}_i$
 - Where softmax($\mathbf{a}_i$) is the importance/weight of the i^{th} word in the sequence.
- **Decoder update:** $\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{y}_{t-1}, \mathbf{c}_t)$
 - Where $\mathbf{s}_{t-1}$ is the previous hidden state, $\mathbf{y}_{t-1}$ is the generated token (or ground truth during training)

4.2 Self-Attention

Self-attention: seq2seq map, all output pos has correspondence to each input pos; contribution of in pos i to opt pos j depend on compatibility between q_j & k_i . Queries, keys, and values come from the same sequence. $\mathbf{x}_i \in \mathbb{R}^{d_{\text{model}}}$; all $\mathbf{W}$ s are learnable.

- $\mathbf{q}_i = \mathbf{W}_Q \mathbf{x}_i \in \mathbb{R}^{d_k}$, $\mathbf{k}_i = \mathbf{W}_K \mathbf{x}_i \in \mathbb{R}^{d_k}$, $\mathbf{v}_i = \mathbf{W}_V \mathbf{x}_i \in \mathbb{R}^{d_v}$
- Q: Looking for; K: other related words; V: actual

content

- **Comparison:** $\alpha(\mathbf{q}, \mathbf{k}_i) = (\mathbf{q}^\top \mathbf{k}_i) / \sqrt{d_k}$. Dot product for performance, $\sqrt{d_k}$ since variance of dot product grows proportionally to d_k .
- **Reduction:** $f(\mathbf{V}, \mathbf{a}) = \text{softmax}(\mathbf{a}) \cdot \mathbf{v}$

Attention for each query is computed independently and in parallel.

In matrix form: $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$:

- $\mathbf{Q} = \mathbf{X}(\mathbf{W}_Q)^\top$, $\mathbf{K} = \mathbf{X}(\mathbf{W}_K)^\top$, $\mathbf{V} = \mathbf{X}(\mathbf{W}_V)^\top$
- $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$, $\mathbf{S} \in \mathbb{R}^{T \times T}$, $\mathbf{A} = \text{softmax}(\mathbf{S})$
- $\mathbf{H} = \mathbf{A}\mathbf{V}$, $\mathbf{H} \in \mathbb{R}^{T \times d_{\text{model}}}$

4.3 Positional Encoding

- Relative key: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = \mathbf{q}_i^\top (\mathbf{k}_j + \mathbf{p}_{i-j}) / \sqrt{d_k}$, $\mathbf{p}_{i-j} \in \mathbb{R}^{d_k}$ learnt
- Rel attn bias: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = \mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + p_{i-j}$, $p_{i-j} \in \mathbb{R}$ learnt
- Rotary pos enc: $\alpha(\mathbf{q}_i, \mathbf{k}_j) = (\mathbf{R}_i \mathbf{q}_i)^\top (\mathbf{R}_j \mathbf{k}_j) / \sqrt{d_k}$, $\mathbf{R}_i$ hardcoded per pos

4.4 MHSA

Compute H attention heads in parallel, then combine them: $\mathbf{H}^{(i)} = \text{self_attention}(\mathbf{W}_Q^{(i)}, \mathbf{W}_K^{(i)}, \mathbf{W}_V^{(i)})$, $i = 1 \dots H$.

output = $\mathbf{W}_O [\mathbf{H}^{(1)} \dots \mathbf{H}^{(H)}]^\top$. Model dimension split evenly among different heads. $d_k = d_v = \frac{d_{\text{model}}}{H}$. For each

head: $\mathbf{Q}^{(i)}, \mathbf{K}^{(i)}, \mathbf{V}^{(i)} \in \mathbb{R}^{T \times \frac{d_{\text{model}}}{H}}$. Each head produces

$\mathbf{H}^{(i)} \in \mathbb{R}^{T \times \frac{d_{\text{model}}}{H}}$ and head outputs are concatenated: $\text{concat}(\mathbf{H}^{(1)}, \dots, \mathbf{H}^{(H)}) \in \mathbb{R}^{T \times d_{\text{model}}}$. Then, apply a final linear projection:

$\text{opt} = \text{concat}(\mathbf{H}^{(1)}, \dots) \mathbf{W}_O$, $\mathbf{W}_O \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. $\mathbf{W}_O$ mixes info across heads and restores original dimension. output $\in \mathbb{R}^{T \times d_{\text{model}}}$

MHA does not increase output dim, but it allows models to:

- Attend to different parts of the sequence at once
- Use different learned projections for each head
- Combine these views into a single representation

Intuition: Each head captures a different pattern in the sequence, and the model learns how to combine them.

5 Transformers

5.1 Layer Normalization

For all $\mathbf{h}_t \in \mathbb{R}^d$, $t \in \{1 \dots T\}$:

- $\mu_t = \frac{1}{d} \sum_{i=1}^d h_{t,i}$, $\sigma_t^2 = \frac{1}{d} \sum_{i=1}^d (h_{t,i} - \mu_t)^2$
- $\text{LN}(\mathbf{h}_t)_i = \gamma_i \frac{h_{t,i} - \mu_t}{\sqrt{\sigma_t^2 + \epsilon}} + \beta_i$

Ensures: normalization stats don't depend on batch. Better than batch norm when batch comp varies over time. Behaves identically during training and inference.

5.2 Position-Wise Feed-Forward Networks

Apply the same MLP independently to each position. A small MLP after attention.

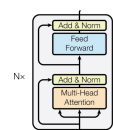
- $\mathbf{h}_t = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}_t + \mathbf{b}_1) + \mathbf{b}_2$

We are applying the same MLP to each vector in each position, that is, $t = 1 \dots T$. Empirically, $\mathbf{W}_1 = \mathbb{R}^{d_{\text{ff}} \times d}$, $d_{\text{ff}} = 4d$, and the 4 is an empirical result. GELU good activation. Don't make them separately, param count should not depend on T . Why? Attention is non-linear. Need 2 b/c expressivity.

5.3 Embeddings and Classifier

- Embedding
- Map token to vector
- Embedding matrix: $\mathbf{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$
- Token $i \rightarrow \text{row } \mathbf{E}_i$
- AFTER, add positional embedding: $\mathbf{x}_t = \mathbf{E}_i + \mathbf{p}_t$
- Output: $\mathbf{X} \in \mathbb{R}^{T \times d}$, which is the "input" to attention
- Classifier:
 - Hidden state at pos t : $\mathbf{h}_t \in \mathbb{R}^d$
 - Want to convert it back to something about the vocabulary: $\mathbf{z}_t = \mathbf{h}_t^\top \mathbf{E}^\top$, back to a vector as large as the vocab size and produce one score for every token in the vocab
 - Logits: $\mathbf{z}_t \in \mathbb{R}^{|\mathcal{V}|}$
 - Softmax $\rightarrow$ next token distribution

5.4 Transformer Layer Architecture



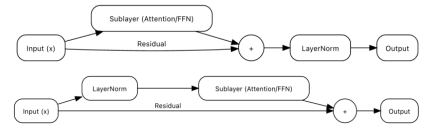
Parameters:

- MHA: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$, each $d_{\text{model}} \times d_{\text{model}}$.
- Feed-forward (w/ act twice): $\mathbf{W}_1 (d_{\text{ff}} \times d_{\text{model}}), \mathbf{W}_2 (d_{\text{ff}} \times d_{\text{model}}), \mathbf{b}_1 (d_{\text{ff}}), \mathbf{b}_2 (d_{\text{model}})$
- d_{model} is the embedding dimension: each word is

represented by this many coordinates.

- LayerNorm, done **twice**, normally after adding residual connections: γ, β (d_{model})

5.5 Pre-Layer Norm



- Bottom is pre-layer norm.
- With it: Gradient has a path not interrupted by LayerNorm
- Residual connections are untouched by LayerNorm

5.6 Bias Removal

Remove bias from equations: $\mathbf{h}_t = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}_t)$.

$\text{LN}(\mathbf{h}_t)_i = \gamma_i \frac{h_{t,i}}{\sqrt{\frac{1}{d} \sum_{j=1}^d (h_{t,j})^2 + \epsilon}}$. Done because it doesn't hurt

5.7 SwiGLU

Originally ReLU in the FFN. Modern models: gated linear units, GLU family. Gating idea:

- Two branches: content and gate
- Gate controls signal element wise
- Improve expressiveness

FFN with SwiGLU: swish is smooth, nonmonotonic, improve grad flow; intermediate dim reduced to 2/3 of original to keep parameter count similar

- $\text{FFN}(\mathbf{x}_t) = \mathbf{W}_3 ((\mathbf{W}_1 \mathbf{x}_t) \odot \text{Swish}(\mathbf{W}_2 \mathbf{x}_t))$
- $\text{Swish}(\mathbf{x}_t) = \mathbf{x}_t \odot \sigma(\mathbf{x}_t)$

5.8 Flash Attention

Flash attention produces the same result as standard attention but uses way less memory.

- Reduce the memory cost of standard self-attention.
- Do not store the full $T \times T$ attention matrix.
- Compute attention in small blocks.
- Keep intermediate results on-chip instead of reading and writing to main memory. (More computation, more memory access but faster memory is accessed).

Benefits:

- Uses much less memory than standard attention.
- Runs much faster in practice.
- Produces the exact same result as standard attention.

5.9 Local Self Attention

Hence, compute attention Q, K, V over a window size W centered on that position. May increase path length – long-range interactions require multiple layers. **Fix:** alternate local & global attention layers

5.10 Grouped Query Attention

Because it's faster (except full MQA BAD PERF!!!) w/ G groups. Most diversity of heads come from queries. Downsides: expressivity



5.11 Encoder-Only Transformation

Encoder only transformers is just the transformer block repeated L times, w/ the output seq of length T . It is meant to do the three:

- **Sequence labeling:** After going through the transformer block, apply a classification head afterward and predict one label per position.
- **Sequence classification:** Reserve the start of the sequence for the label. Process sequence as usable, extract final reserved label state
- **Span extraction:** Task is to retrieve a slice $a : b$ that is the answer to a question. Todo: concat Q and passage (full text). Predict a start and end score for each token. Select highest storing valid start-end pair.

5.12 Pre-Training

Done because of the inductive biases of transformers. Some models learn better when data is a specific type of pattern. Since transformers inductive biases are weak, learning requires lots of data. Patterns best learned from large unlabeled data. Pretraining is like initialization. Post-train: make model ready 4 specific task

Fine tuning: use a labelled dataset to steer the model towards the right behavior.

5.13 How to Pre-Train

Inductive biases are the set of assumptions the model makes about the data.

Transformers inductive bias: every output position depends on every input position. **Very weak bias.** Ways of pre-training? (no need for labels)

- **Autoregressive:** Model each token based on previous
 - **MLM:** Make a hole, predict the hole
 - **BERT:** Make 15% of holes, predict them
- These are self-supervised. Labels come from input.